Selective State Space Models: Exercise Set

NeurIPS 2026 Education Track

Five exercises covering Notebooks 1–3 (discretization, selectivity, parallel scan). Solutions are provided in a separate document. Exercises 1 and 4 are hand computations; 2, 3, and 5 require you to run or modify the accompanying notebooks.

Exercise 1 — Discretization by hand

Consider the scalar continuous-time system

$$h'(t) = a\,h(t) + b\,x(t), \qquad a = -4, \quad b = 2.$$

(a) Derive the zero-order hold (ZOH) discretization formulas

$$\bar{a} = e^{a\Delta}, \qquad \bar{b} = \frac{\bar{a} - 1}{a}\,b$$

starting from the assumption that $x(t) = x_k$ is held constant over $t \in [k\Delta, (k+1)\Delta)$. Show your integration steps.

(b) Using your formulas, compute $\bar{a}$ and $\bar{b}$ numerically for $\Delta = 0.25$. Give both to 4 decimal places.

(c) Compute $\bar{a}$ and $\bar{b}$ again for $\Delta = 2.0$. Is the system still stable (i.e. does $|\bar{a}| < 1$) at this larger step size? Explain why or why not, referencing the sign and magnitude of a.

(d) Verify your part (b) answer against `discretize_zoh(a, b, dt)` from Notebook 1.

Exercise 2 — Predicting tensor shapes

In the `SelectiveSSM` block from Notebook 2, suppose:

- batch size $B = 16$
- sequence length $L = 30$
- input dimension `in_dim` $= 4$
- state dimension `state_dim` $= 12$

For each of the following tensors computed inside `forward()`, give its exact shape:

(a) `A` (after `-torch.exp(self.A_log)`)

(b) `Bt = self.in_proj(x)`

(c) `Ct = self.c_proj(x)`

(d) `delta` (after the softplus)

(e) `a_bar` and `b_bar` *inside* the loop, at a single timestep `t`

(f) `h`, both before the loop starts and inside the loop

(g) the final returned tensor `y`

For each answer, state which line of shape-changing code justifies it (broadcasting, matrix multiply, or a named PyTorch op).

Exercise 3 — Breaking selective copying

The selective-copying task in Notebook 2 has the model output the *most recent* signal value at every step. Consider this variant instead: the target at every step is the **running average of all signal values seen so far** (not just the most recent one).

(a) Modify `make_batch` to produce this new target. (Hint: you need to track a running sum and a running count of markers seen, not just `last_val`.)

(b) Before running anything: predict, in 3–5 sentences, whether the `SelectiveSSM` architecture as given (with a single scalar Δ_t gate per state channel) can represent a running average exactly, or only approximately. Justify your answer in terms of what operation a state update $h_t = \bar{a}_t h_{t-1} + \bar{b}_t$ can and cannot compute.

(c) Train both `SelectiveSSM` and `FixedSSM` on the new task and report final losses. Does the result match your prediction from (b)?

(d) What is the *minimum* change to the architecture (not the task) that would let the model compute an exact running average? (Hint: think about what $\bar{a}_t$ would need to equal.)

Exercise 4 — Scan depth and padding

Recall the doubling parallel scan from Notebook 3: at each round, every element combines with the element 2^k positions before it, for $k = 0, 1, 2, \ldots$ until $2^k \geq n$, where n is the (padded) sequence length including the folded-in initial condition.

(a) For a raw input sequence of length $L = 100$ (so $n = 101$ after folding in h_0), what is the padded length `padded_n` used by `parallel_scan`? Show the power-of-two computation.

(b) How many rounds of the `while step < padded_n` loop execute? Give the exact number, and the general formula in terms of n.

(c) For $L = 1000$ and for $L = 1{,}000{,}000$, compute the number of rounds. Compare the growth rate to L itself and explain in one sentence why this matters for scaling to long sequences.

(d) Sequential scan does exactly L sequential steps. At what value of L does the parallel scan's round count first drop below 1% of L?

Exercise 5 — SSM vs. attention: compute and memory

Let L be sequence length, d be model/embedding dimension, and N be SSM state dimension (typically $N \ll d$).

(a) Write the FLOPs required for one forward pass of (i) self-attention over the full sequence and (ii) a selective SSM layer processed via parallel scan, both as a function of L, d, and (for the SSM) N.

(b) Write the memory required to store activations needed for a single autoregressive decoding step at inference time, for both architectures. For attention, account for the KV cache; state its size as a function of L (context length so far) and d.

(c) Using your formulas from (a) and (b), find the crossover sequence length L^* at which SSM inference memory per step becomes smaller than attention's KV-cache memory, in terms of N and d. (Give a symbolic expression, then evaluate it numerically for $N = 64$, $d = 4096$.)

(d) Name one task property (not a general appeal to "long context") for which this crossover analysis *overstates* the practical advantage of SSMs, referencing the exact-recall limitation discussed on Slide 15 of the accompanying deck.